

AccessPath

Navigation designed
around accessibility, not
just distance.

A mobile navigation app that routes wheelchair users along paths they
can actually use — and tells them when conditions change mid-trip.

- ↙ Step-free
- ↙ Low incline
- 🚶 Accessible entrance
- ⚠ Elevator out

DESIGNER	FOCUS	DELIVERABLE	SCOPE
Payton Nguyen	Mobility / wheelchair routing	3 screens + rationale	Concept, solo, ~1 week

The fastest route and the usable route are not the same route

PROBLEM	Most navigation apps optimize for speed and distance. The fastest route is not always physically usable — one flight of stairs makes it impossible, not slower. Accessibility also changes without warning: elevators break, construction appears, entrances get blocked.
WHO IT'S FOR	A wheelchair user navigating an unfamiliar campus, neighborhood, or transit hub. The gap isn't the person — it's that the physical condition of a route is information nobody is showing them until they're standing in front of it.
USER NEED	“I need to know whether a route works for me before I'm already there.”
DESIGN GOAL	Help users understand whether a route is accessible <i>before</i> they encounter an obstacle, and make it easy to adapt when conditions change.
OUT OF SCOPE	I focused on mobility and wheelchair routing rather than covering mobility, vision, and hearing at once. Vision and hearing support are sketched as future work on the last page.

WHAT I ASSUMED, AND WHAT I DIDN'T DO

I did not run user interviews or collect data for this project, so there are no research findings here. The framing comes from the prompt and my own reasoning. Three assumptions drive the design:

- **Needs vary.** Two wheelchair users won't accept the same route — one avoids all inclines, another doesn't mind a gentle grade. So routing has to be adjustable, not one “accessible mode.”
- **Accessibility data goes stale.** A route that was step-free last month may be blocked today, so recency matters more than volume of reports.
- **Users want to decide.** Hiding the faster route would take the choice away. Showing it with a specific reason keeps the decision with the user.

Ask what “accessible” means to this user, first

Before searching, the user sets what the route should optimize for. These preferences carry through every screen that follows.

9:41

How should we route you?

Pick what matters for your route. You can change this anytime.

From
Sproul Plaza · Current location

Where are you going?
Doe Library

ROUTE PREFERENCES

-  **Step-free route**
Never route me over stairs ☒
-  **Avoid steep inclines**
Keep grades gentle where possible ☒
-  **Accessible entrances**
Ramp or level entry at the door ☒
-  **Elevators preferred**
Use elevators instead of long ramps ☐

Find accessible route

Preferences come before the search, not after it

In most apps, accessibility is a checkbox buried in settings. Putting it on the first screen makes it the premise of the search rather than a filter applied to results.

Each toggle names a physical condition

“Step-free route” and “Avoid steep inclines” instead of one vague “accessible mode” switch. The user knows exactly what is being filtered on, which also makes it clear later *why* a route was rejected.

Defaults on, but all four are optional

Three are on by default because they're the most common needs, and “Elevators preferred” is off because some users specifically avoid them. Nothing is locked — this is the user's definition, not the app's.

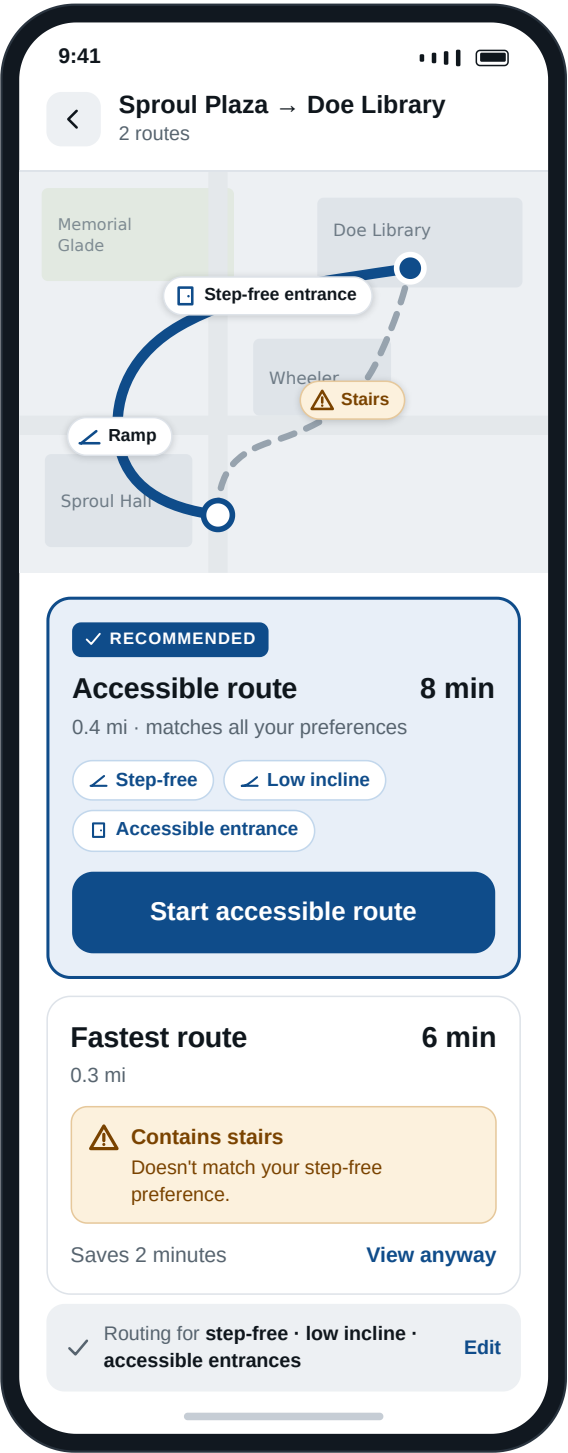
Accessibility of the UI itself

16 px minimum text, 64 px-tall tap targets, an icon and a label on every option, and state shown by the switch position rather than color alone.

Screen 1 — preferences. The toggles work; try them.

Show the trade-off instead of hiding the faster route

The main screen. Two minutes is the entire cost of the accessible route, and the reason the faster one was not recommended is stated in the user's own terms.



Screen 2 — route comparison

The recommended route wins on emphasis, not by deletion

The accessible route gets the filled card, the badge, and the only primary button. The fastest route stays fully readable right below it. Removing it would make the app decide for the user; this way the app makes a recommendation and shows its reasoning.

The warning is specific and checkable

“Contains stairs — doesn't match your step-free preference” instead of “may not be accessible.” It names the physical obstacle and ties it back to the setting the user chose on Screen 1, so the recommendation is traceable rather than a black box.

The cost is stated plainly: 2 minutes

Both times sit at the same size in the same position, and “Saves 2 minutes” names the trade-off out loud. The principle behind the whole screen: usability of a route matters more than its length.

Map markers pair an icon with a word

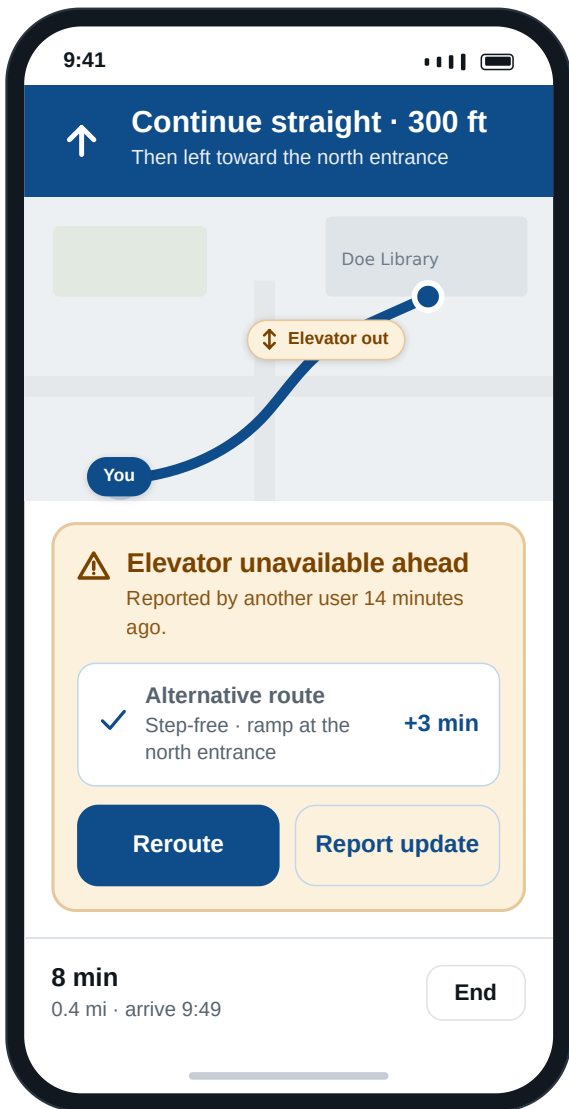
Ramp, step-free entrance, and stairs are labeled, and the two route lines differ in weight and dash pattern as well as color — so nothing on the map depends on color vision to read.

ASSUMPTION

Route details are illustrative. A real version depends on sidewalk grade, curb-cut, and entrance data that is patchy in most cities — which is the genuinely hard part of building this.

A route can stop being accessible after you've started walking it

Static map data can't know that an elevator went out this morning. The alert arrives while the user still has room to change course.



Screen 3 — live obstacle and reroute

The report's age is part of the alert, not a detail

"14 minutes ago" is what makes the report worth acting on. For temporary obstacles, recency is the credibility signal — a report from last week would need to be presented much more tentatively, or not at all.

The alternative is priced before the user commits

+3 minutes and "step-free" appear on the same card as the Reroute button. Same principle as Screen 2: the app recommends, and shows the cost, so accepting the detour is an informed choice rather than a surprise.

Reporting supports navigation — it isn't the product

"Report update" is a secondary button that appears in context, at the moment the user is standing in front of the obstacle and actually knows the answer. There's no separate feed, profile, or points system.

Alert, not interruption

The card slides under the turn instruction rather than covering the map, and it can be ignored — navigation continues either way. An accessibility warning that blocks the screen is its own kind of obstacle.

Three decisions the whole product rests on

Personalized routing

Accessibility needs aren't identical, so the user sets what the route optimizes for instead of accepting one built-in definition of “accessible.”

Accessibility over speed

The interface explains why the recommended route is longer and keeps the faster option visible with a specific reason attached, so the choice stays with the user.

Real-time information

Recent reports cover what static map data misses — broken elevators, construction, closed entrances — and reroute before the user reaches them.

WHAT I CONSIDERED AND REJECTED

One “accessible mode” switch

Simpler than four toggles, but it hides what's actually being filtered on — and a single switch can't explain why a route was rejected.

Four named conditions cost one screen and make every later warning traceable.

Hiding inaccessible routes

A shorter, cleaner list. But it quietly removes the user's judgement — some people would take two steps with a hand, others never would.

Show the route, name the obstacle, let the user decide.

A full crowdsourced access map

More ambitious, but it only works with a dense reporting community, and it turns the product into contributing rather than getting somewhere.

Reporting stays one button, inside navigation.

HOW THE UI APPLIES THIS TO ITSELF

- **Icon plus text, always.** No status is carried by color alone.
- **Large targets and type.** 16 px minimum body text, tap targets above 44 px.
- **Plain language.** “Contains stairs,” not “limited accessibility.”
- **Strong contrast, low clutter.** One accent color, one warning color, no decorative gradients.

WHERE I'D TAKE IT NEXT

- **Vision.** Spoken directions cued to landmarks and surfaces, not just distances.
- **Hearing.** Visual and haptic transit alerts for stops and platform changes.
- **Verified venues.** Operator-confirmed entrance and elevator data alongside user reports.
- **The real blocker.** Sourcing sidewalk grade, curb-cut, and entrance data — and getting enough active reporters in one place for live updates to be dependable.

AccessPath is a concept project made for the UX@Berkeley Fall 2026 application. It is not a shipping product, and no user research or data collection was conducted for it — every judgement here is my own reasoning from the prompt, and the assumptions behind it are labelled on page 02.